In-Class Activity 5: Logistic Regression and Support Vector Machines

GEOL 599: Machine Learning

```
# If needed, install these packages first:
# install.packages(c("tidyverse", "tidymodels", "kernlab"))
#install.packages("kernlab")
library(tidyverse)
library(tidymodels)
library(kernlab)

tidymodels_prefer()
set.seed(30)
```

Overview

In this activity, you will compare **logistic regression** and **support vector machines (SVMs)** using the Palmer Penguins dataset.

The dataset contains field measurements from penguins observed in Antarctica. We will use two bill measurements to classify penguin species and visualize how different SVM kernels change the decision boundary.

Learning Goals

By the end of this activity, you should be able to:

- Fit a logistic regression classifier with `tidymodels`.
- Fit SVM classifiers with linear, polynomial, and radial basis function kernels.
- Explain why feature scaling matters for SVMs.
- Compare model performance using accuracy, confusion matrices, precision, recall, and F1-score.
- Visualize how different SVM kernels change the decision boundary.

Load the Dataset

We will use the Palmer Penguins dataset. The dataset is loaded from a CSV file.

```
data_url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/penguins.csv"

penguins <- read_csv(data_url, show_col_types = FALSE)

glimpse(penguins)
```

```
## Rows: 344
## Columns: 8
## $ species           <chr> "Adelie", "Adelie", "Adelie", "Adelie", "Adelie", "Adelie", "Adelie", "Adel
## $ island            <chr> "Torgersen", "Torgersen", "Torgersen", "Torgersen", "Torgersen", "Torgerse
```

```
## $ bill_length_mm    <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, 42.0, 37.8, 37.8, 41.1
## $ bill_depth_mm     <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, 20.2, 17.1, 17.3, 17.6
## $ flipper_length_mm <dbl> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186, 180, 182, 191, 198,
## $ body_mass_g       <dbl> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, 4250, 3300, 3700, 3200
## $ sex               <chr> "male", "female", "female", NA, "female", "male", "female", "male", NA, NA
## $ year              <dbl> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 20
```

Prepare a Binary Classification Problem

To make the decision boundaries easy to visualize, we will classify only two species:

- Chinstrap
- Gentoo

We will use two predictors:

- bill_length_mm
- bill_depth_mm

This is a simplified model. In a real analysis, we might use more predictors, but using two features lets us plot the decision boundary directly.

```
species_to_keep <- c("Chinstrap", "Gentoo")

penguins_binary <- penguins %>%
  filter(species %in% species_to_keep) %>%
  select(species, bill_length_mm, bill_depth_mm) %>%
  drop_na() %>%
  mutate(species = factor(species, levels = species_to_keep))

penguins_binary %>%
  slice_head(n = 6)
```

```
## # A tibble: 6 x 3
##   species bill_length_mm bill_depth_mm
##   <fct>            <dbl>         <dbl>
## 1 Gentoo            46.1          13.2
## 2 Gentoo            50            16.3
## 3 Gentoo            48.7          14.1
## 4 Gentoo            50            15.2
## 5 Gentoo            47.6          14.5
## 6 Gentoo            46.5          13.5
```

Question 1

How many observations are in each class? Is the dataset balanced? There are 191 rows in the binary penguin dataset with 68 Chinstrap and 123 Gentoo observations. The dataset is unbalanced roughly a 1:2 Chinstrap:Gentoo ratio.

Hint: Use `count()` on the target column.

```
# TODO: count the number of observations in each species class
length(penguins_binary$species)
```

```
## [1] 191
```

```
table(penguins_binary$species)
```

```
##
## Chinstrap    Gentoo
##        68       123
```

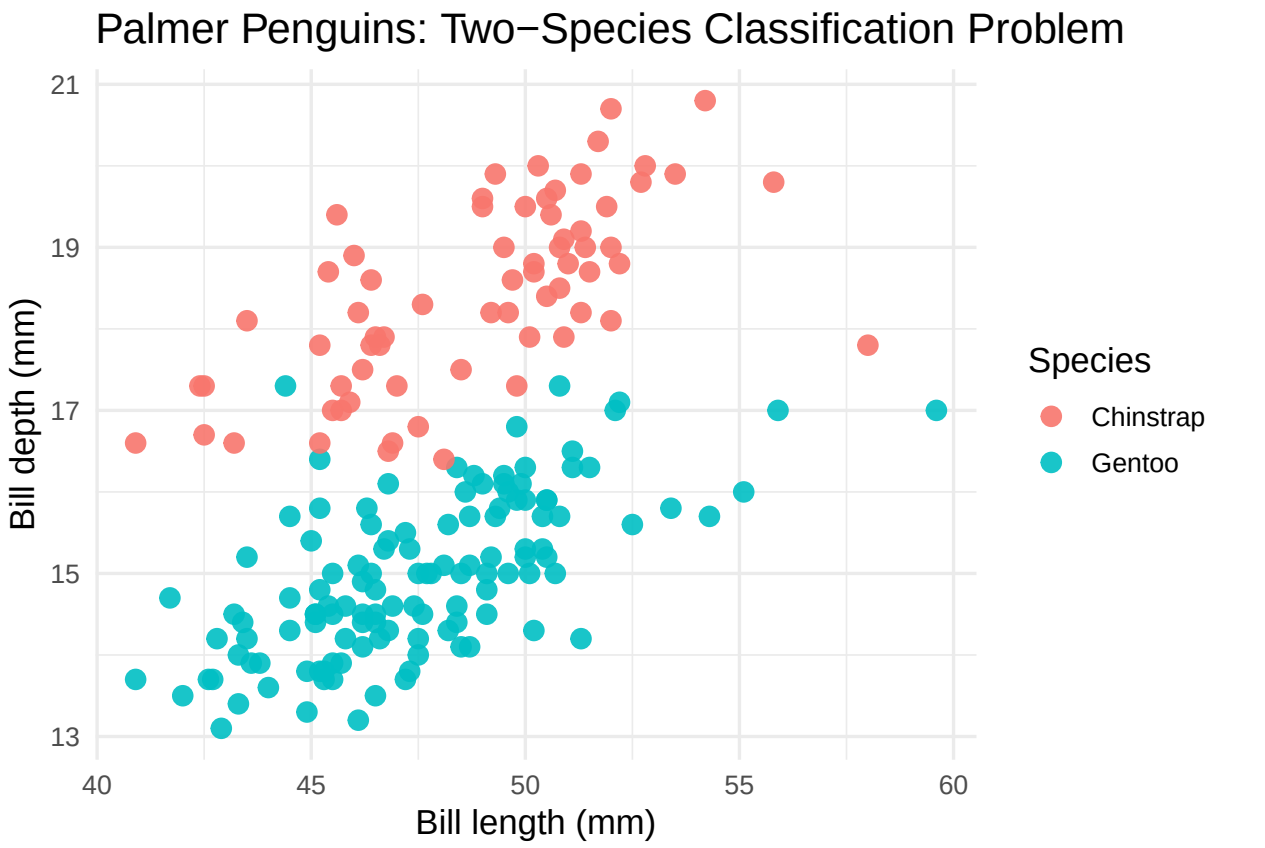
```
68/123
```

```
## [1] 0.5528455
```

Visualize the Data

Before fitting models, plot the two predictors. This helps us anticipate whether a linear boundary might work.

```
ggplot(penguins_binary, aes(x = bill_length_mm, y = bill_depth_mm, color = species)) +
  geom_point(size = 3, alpha = 0.9) +
  labs(
    title = "Palmer Penguins: Two-Species Classification Problem",
    x = "Bill length (mm)",
    y = "Bill depth (mm)",
    color = "Species"
  ) +
  theme_minimal(base_size = 13)
```



Question 2

Based on the scatterplot, do you expect a linear classifier to work well? Why or why not? I would expect a linear clasifier to work well because visually it looks like I could draw a straight line between the two groups and classify maybe less than 10 wrong.

Train/Test Split

Now split the data into training and testing sets.

Use `strata = species` so that both the training and testing sets preserve the class proportions.

```
# TODO: create a train/test split
# Use prop = 0.75 and strata = species

penguin_split <- initial_split(penguins_binary, prop = 0.75, strata = 'species')

train_data <- training(penguin_split)
test_data <- testing(penguin_split)

train_data %>% count(species)
```

```
## # A tibble: 2 x 2
##   species       n
##   <fct>     <int>
## 1 Chinstrap    51
## 2 Gentoo       92
```

```
test_data %>% count(species)
```

```
## # A tibble: 2 x 2
##   species       n
##   <fct>     <int>
## 1 Chinstrap    17
## 2 Gentoo       31
```

Preprocessing Recipe

SVMs are sensitive to feature scale, so we will normalize the numeric predictors. Logistic regression also benefits from this when we want to compare coefficients or model behavior across predictors.

```
penguin_recipe <- recipe(species ~ bill_length_mm + bill_depth_mm, data = train_data) %>%
  step_normalize(all_numeric_predictors())

penguin_recipe
```

```
##
## -- Recipe ----------------------------------------------------------------------------------------
##
## -- Inputs
## Number of variables by role
## outcome:   1
## predictor: 2
##
## -- Operations
## * Centering and scaling for: all_numeric_predictors()
```

Define the Models

We will compare four classifiers:

1. Logistic regression
2. Linear SVM
3. Polynomial-kernel SVM
4. RBF-kernel SVM

In R, these models are specified with `parsnip` and then combined with the preprocessing recipe using a `workflow()`.

```
model_specs <- list(
  "Logistic regression" = logistic_reg() %>%
    set_mode("classification") %>%
    set_engine("glm"),

  "Linear SVM" = svm_linear(cost = 1) %>%
    set_mode("classification") %>%
    set_engine("kernlab"),

  "Polynomial SVM" = svm_poly(cost = 1, degree = 3, scale_factor = 1) %>%
    set_mode("classification") %>%
    set_engine("kernlab"),

  "RBF SVM" = svm_rbf(cost = 1, rbf_sigma = 0.1) %>%
    set_mode("classification") %>%
    set_engine("kernlab")
)
```

Fit and Evaluate the Models

Fit each model on the training data and evaluate it on the test data.

```
fit_one_model <- function(model_spec) {
  workflow() %>%
    add_recipe(penguin_recipe) %>%
    add_model(model_spec) %>%
    fit(data = train_data)
}

model_fits <- map(model_specs, fit_one_model)
```

```
test_predictions <- imap_dfr(
  model_fits,
  function(fitted_workflow, model_name) {
    predict(fitted_workflow, test_data, type = "class") %>%
      bind_cols(test_data %>% select(species)) %>%
      mutate(model = model_name)
  }
)

test_predictions %>%
  slice_head(n = 10)
```

```
## # A tibble: 10 x 3
```

```
##    .pred_class species model
##    <fct>       <fct>   <chr>
##  1 Gentoo      Gentoo  Logistic regression
##  2 Gentoo      Gentoo  Logistic regression
##  3 Gentoo      Gentoo  Logistic regression
##  4 Gentoo      Gentoo  Logistic regression
##  5 Gentoo      Gentoo  Logistic regression
##  6 Gentoo      Gentoo  Logistic regression
##  7 Gentoo      Gentoo  Logistic regression
##  8 Gentoo      Gentoo  Logistic regression
##  9 Gentoo      Gentoo  Logistic regression
## 10 Gentoo      Gentoo  Logistic regression
```

```
accuracy_results <- test_predictions %>%
  group_by(model) %>%
  accuracy(truth = species, estimate = .pred_class) %>%
  arrange(desc(.estimate))

accuracy_results
```

```
## # A tibble: 4 x 4
##   model               .metric  .estimator .estimate
##   <chr>               <chr>    <chr>          <dbl>
## 1 Linear SVM          accuracy binary         0.958
## 2 Logistic regression accuracy binary         0.958
## 3 Polynomial SVM      accuracy binary         0.938
## 4 RBF SVM             accuracy binary         0.938
```

Question 3

Which model has the highest test accuracy? Are the differences large enough to be meaningful? The linear SVM and logistic regression estimate was 95.8% accurate, and the polynomial SVM and RBF SVM were 93.8% accurate. These differences are not large enough to be meaningful.

Confusion Matrices

Accuracy is useful, but it does not show which classes are being confused. Create a confusion matrix for each model.

```
confusion_data <- test_predictions %>%
  group_by(model) %>%
  conf_mat(truth = species, estimate = .pred_class)

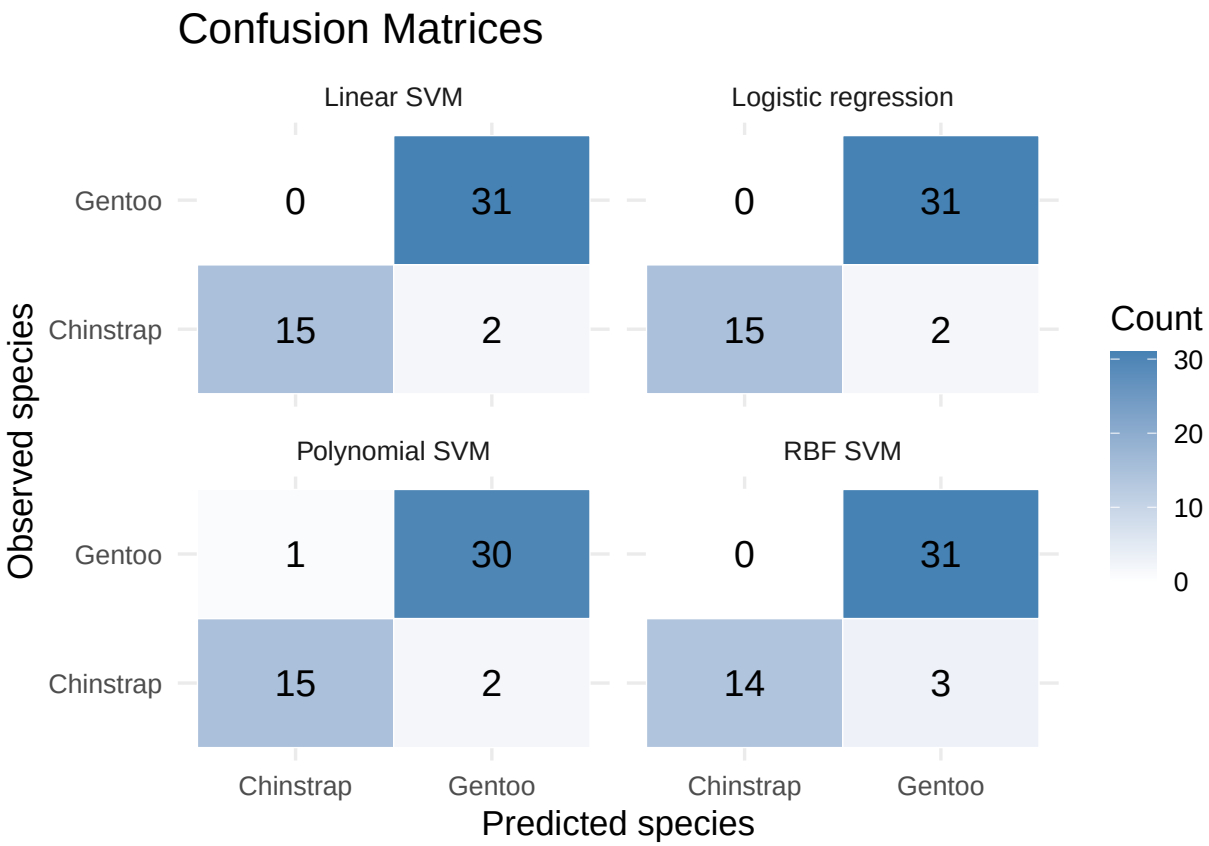
confusion_data
```

```
## # A tibble: 4 x 2
##   model               conf_mat
##   <chr>               <list>
## 1 Linear SVM          <conf_mat>
## 2 Logistic regression <conf_mat>
## 3 Polynomial SVM      <conf_mat>
## 4 RBF SVM             <conf_mat>
```

```
confusion_plot_data <- test_predictions %>%
  count(model, species, .pred_class, name = "n") %>%
```

```
  complete(model, species, .pred_class, fill = list(n = 0))

ggplot(confusion_plot_data, aes(x = .pred_class, y = species, fill = n)) +
  geom_tile(color = "white") +
  geom_text(aes(label = n), size = 5) +
  facet_wrap(~ model) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(
    title = "Confusion Matrices",
    x = "Predicted species",
    y = "Observed species",
    fill = "Count"
  ) +
  theme_minimal(base_size = 13)
```



Question 4

Do any models make noticeably different types of mistakes? Explain using the confusion matrices. These confusion matricies are all very similar. The polynomial SVM miscalssifies 1 Gentoo and the RBF misclassifies 3 Chinstraps compared to the other models' 2.

Classification Metrics

A classification summary gives metrics such as precision, recall, and F1-score.

```
classification_metrics <- metric_set(accuracy, precision, recall, f_meas)
```

```r
class_metric_results <- test_predictions %>%
  group_by(model) %>%
  classification_metrics(truth = species, estimate = .pred_class)

class_metric_results
```

```
## # A tibble: 16 x 4
##    model               .metric   .estimator .estimate
##    <chr>               <chr>     <chr>          <dbl>
##  1 Linear SVM          accuracy  binary         0.958
##  2 Logistic regression accuracy  binary         0.958
##  3 Polynomial SVM      accuracy  binary         0.938
##  4 RBF SVM             accuracy  binary         0.938
##  5 Linear SVM          precision binary         1
##  6 Logistic regression precision binary         1
##  7 Polynomial SVM      precision binary         0.938
##  8 RBF SVM             precision binary         1
##  9 Linear SVM          recall    binary         0.882
## 10 Logistic regression recall    binary         0.882
## 11 Polynomial SVM      recall    binary         0.882
## 12 RBF SVM             recall    binary         0.824
## 13 Linear SVM          f_meas    binary         0.938
## 14 Logistic regression f_meas    binary         0.938
## 15 Polynomial SVM      f_meas    binary         0.909
## 16 RBF SVM             f_meas    binary         0.903
```

Visualize Decision Boundaries

The function below draws the classification regions learned by each model.

The background color shows the predicted class across the feature space. The points show the actual observations.

- Circles = training data
- X symbols = test data
- Point color = observed species
- Background color = model-predicted species

```r
plot_decision_boundary <- function(fitted_workflow, train_data, test_data, title) {
  x_min <- min(train_data$bill_length_mm, test_data$bill_length_mm) - 1
  x_max <- max(train_data$bill_length_mm, test_data$bill_length_mm) + 1
  y_min <- min(train_data$bill_depth_mm, test_data$bill_depth_mm) - 1
  y_max <- max(train_data$bill_depth_mm, test_data$bill_depth_mm) + 1

  prediction_grid <- expand_grid(
    bill_length_mm = seq(x_min, x_max, length.out = 250),
    bill_depth_mm = seq(y_min, y_max, length.out = 250)
  )

  grid_predictions <- predict(fitted_workflow, prediction_grid, type = "class") %>%
    bind_cols(prediction_grid)

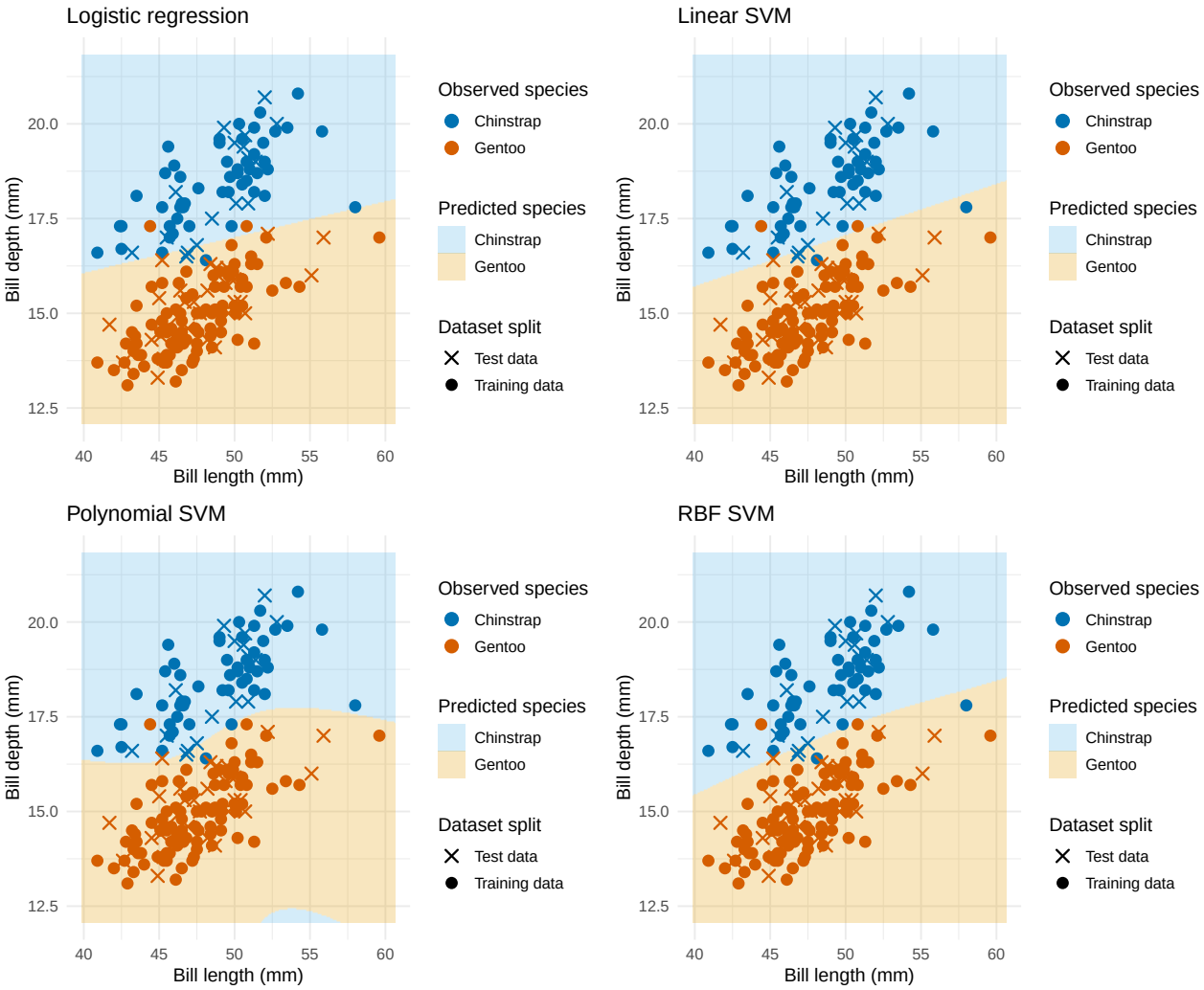
  points_data <- bind_rows(
    train_data %>% mutate(split = "Training data"),
    test_data %>% mutate(split = "Test data")
```

```
  )

  ggplot() +
    geom_raster(
      data = grid_predictions,
      aes(x = bill_length_mm, y = bill_depth_mm, fill = .pred_class),
      alpha = 0.25
    ) +
    geom_point(
      data = points_data,
      aes(x = bill_length_mm, y = bill_depth_mm, color = species, shape = split),
      size = 3,
      stroke = 1
    ) +
    scale_fill_manual(values = c("Chinstrap" = "#56B4E9", "Gentoo" = "#E69F00")) +
    scale_color_manual(values = c("Chinstrap" = "#0072B2", "Gentoo" = "#D55E00")) +
    scale_shape_manual(values = c("Training data" = 16, "Test data" = 4)) +
    labs(
      title = title,
      x = "Bill length (mm)",
      y = "Bill depth (mm)",
      fill = "Predicted species",
      color = "Observed species",
      shape = "Dataset split"
    ) +
    coord_cartesian(xlim = c(x_min, x_max), ylim = c(y_min, y_max)) +
    theme_minimal(base_size = 12)
}
```

```
decision_boundary_plots <- imap(
  model_fits,
  ~ plot_decision_boundary(.x, train_data, test_data, .y)
)

# Display the four plots together.
# If patchwork is not installed, run install.packages("patchwork") or print each plot one at a time.
if (requireNamespace("patchwork", quietly = TRUE)) {
  library(patchwork)
  wrap_plots(decision_boundary_plots, ncol = 2)
} else {
  walk(decision_boundary_plots, print)
}
```



Question 5

Compare the decision boundaries.

- Which models produce straight decision boundaries? Logisitic and linear.
- Which models produce curved decision boundaries? Polynomial and RBF, the RBF barely looks curved,
- Which model seems most appropriate for this dataset? Logistic

Kernel Exploration

Now experiment with SVM settings.

Try changing:

- `cost`
- `degree` for the polynomial kernel
- `rbf_sigma` for the RBF kernel

Question 6

Choose one SVM kernel and change one parameter. What changed in the decision boundary?

Hint: Smaller `cost` usually allows a wider margin with more tolerance for mistakes. Larger `cost` tries harder to classify the training data correctly. For RBF kernels, larger `rbf_sigma` can create more local, flexible decision boundaries.

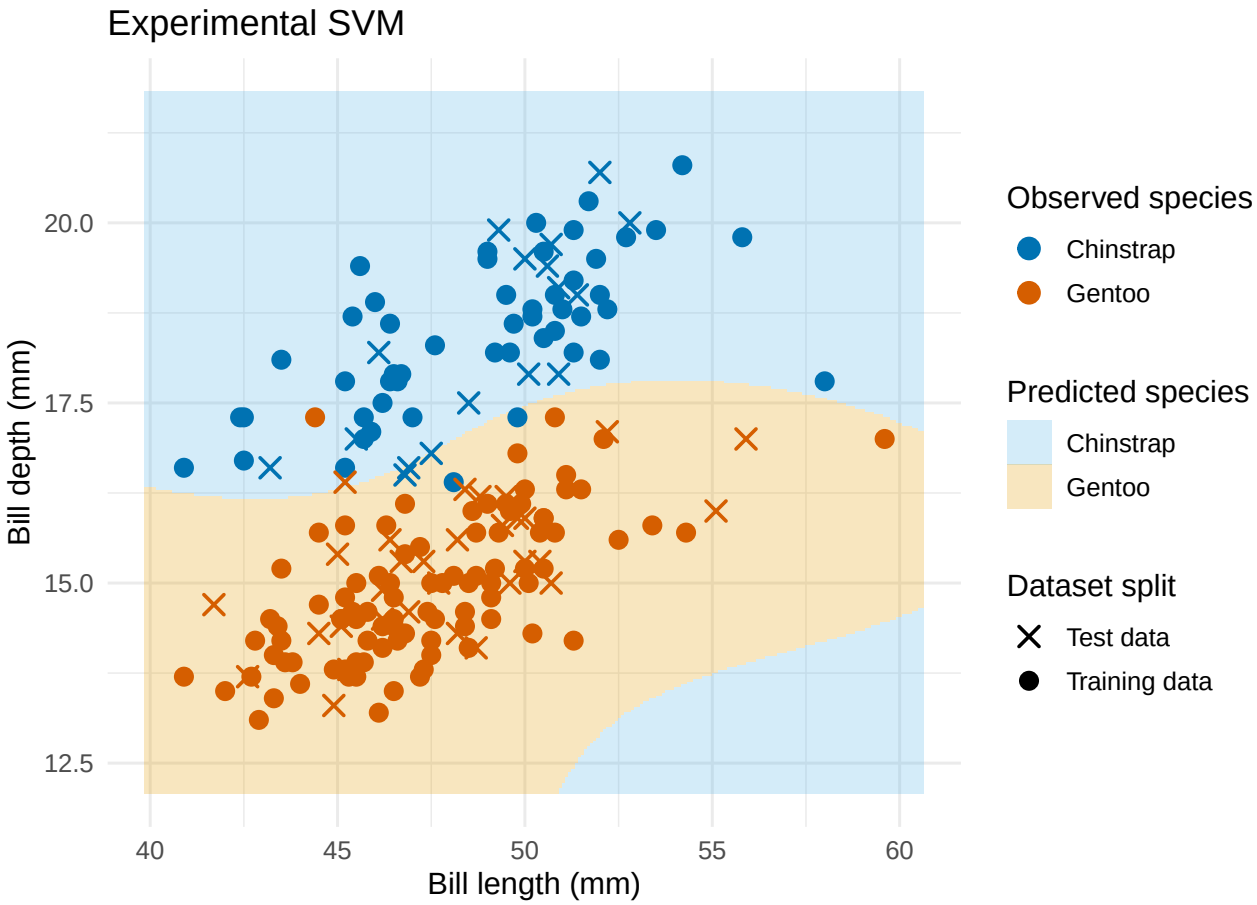
```
# TODO: modify one SVM model below and visualize its decision boundary

experiment_spec <-  svm_poly(cost = 2, degree = 3, scale_factor = 1) %>%
    set_mode("classification") %>%
    set_engine("kernlab")



experiment_fit <- workflow() %>%
  add_recipe(penguin_recipe) %>%
  add_model(experiment_spec) %>%
  fit(data = train_data)

plot_decision_boundary(
  experiment_fit,
  train_data,
  test_data,
  "Experimental SVM"
)
```




```
experiment_predictions <- predict(experiment_fit, test_data, type = "class") %>%
  bind_cols(test_data %>% select(species))
```

```r
experiment_predictions %>%
  accuracy(truth = species, estimate = .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.917
```

Reflection

Answer the following questions in a few sentences each.

1. Why do we scale predictors before fitting an SVM? SVMs are drawing a boundary that is sensitive to the distance points are away from that boundary. Scaling is needed so that the influence of those distances can be measured consistently.
2. How is logistic regression different from a linear SVM? A logistic regression is affected by all observations, and a linear SVM is only affected by the support vectors. A logistic regression outputs probabilities and SVMs have classified outputs. SVMs can only use L2 regularizaiton and logistic regressions can use L2 and L1.
3. What does the kernel do in an SVM? A kernel quantifies the similarity between observations. It is used to allow for non-linear relationships between observations.
4. Why might a highly flexible kernel overfit? A highly flexible kernel could classify similarities that aren't useful for the model to learn in a general sense. Then when it is extrapolated to other data, those patterns the highly flexible kernel used might not exist.
5. In environmental science, when might a nonlinear decision boundary be useful? For data that has more mixed outputs or more variables, nonlinear boundary may be more useful. If one of the classes was more nested within another class like a suitable range of salinity for critters a more complex line would be needed.